

SYSTEM DESIGN BLUEPRINT

Secure AI-Powered Contract Management Workspace

Version 1.0 | University Final Project | Team of 3 Developers

Prepared by: Senior Software Architect

June 11, 2026

A. High-Level System Architecture

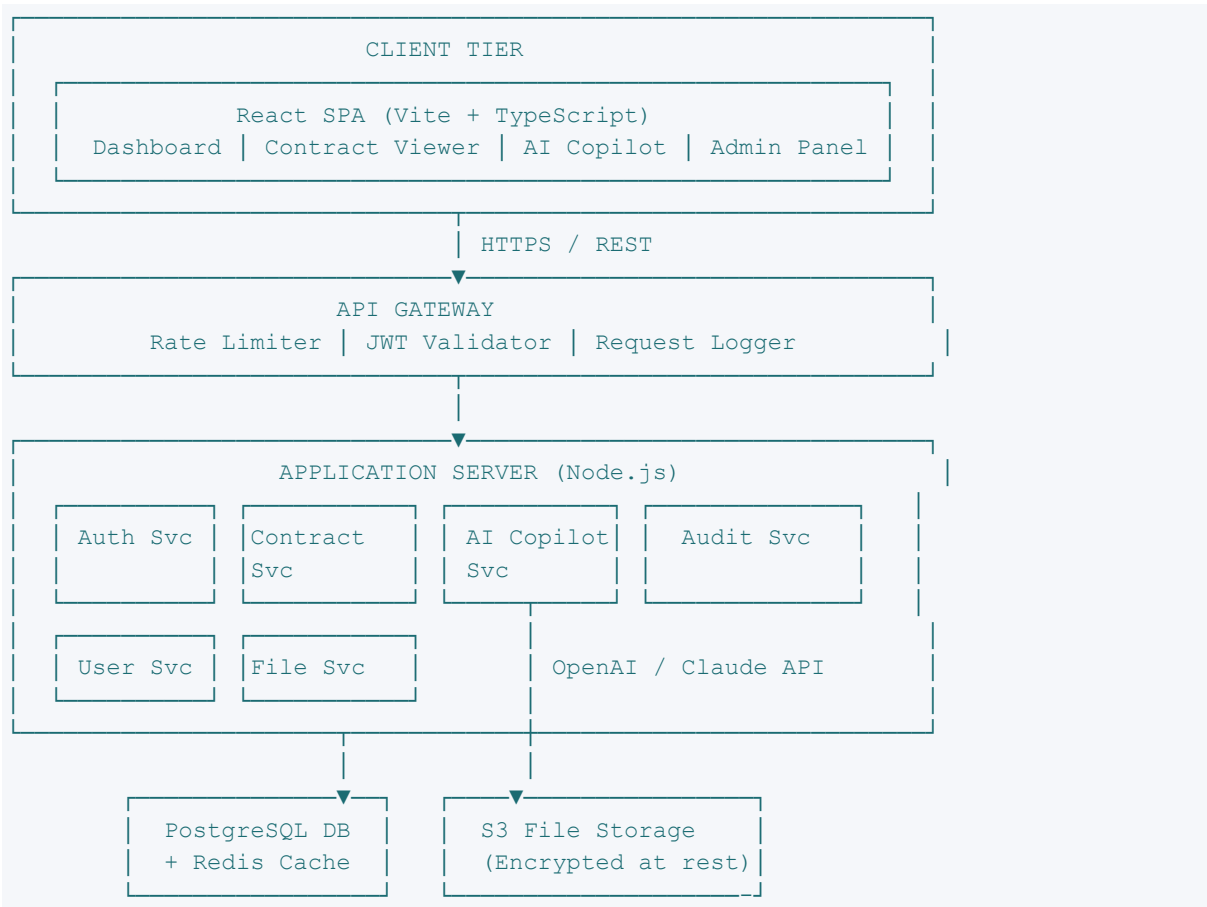
The platform follows a layered, microservice-inspired monolith architecture. All components communicate through well-defined service interfaces, enabling a future migration to microservices without restructuring the codebase.

A.1 Architecture Overview

The system is composed of five primary layers:

- Frontend (React SPA) — User interface served via CDN
- API Gateway — Single entry point, rate limiting, JWT validation
- Backend Application Server (Node.js / Express) — Business logic, services, and controllers
- Data Layer — PostgreSQL (relational data) + Redis (sessions/cache)
- AI Service — LLM integration via OpenAI/Anthropic API
- File Storage — S3-compatible object storage with server-side encryption

A.2 Architecture Diagram



A.3 Technology Stack

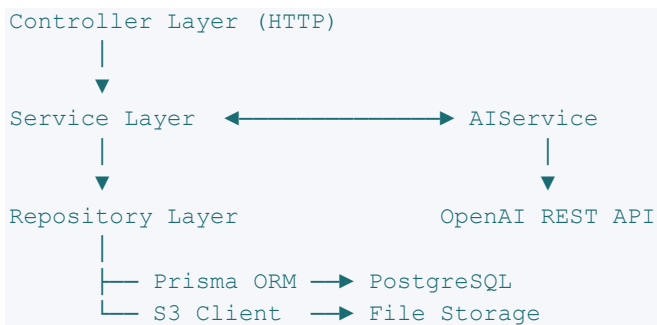
Layer	Technology	Rationale
Frontend	React 18, TypeScript, Vite, Tailwind CSS	Type safety, fast builds, utility-first styling
State Management	Zustand + React Query	Lightweight global state + server state caching
Backend	Node.js 20, Express 5, TypeScript	Uniform language across stack, large ecosystem
ORM	Prisma	Type-safe queries, migration management
Database	PostgreSQL 15	ACID compliance, JSON support, row-level security
Cache / Sessions	Redis 7	Fast session lookup, token blacklisting
File Storage	AWS S3 (or MinIO for local)	Scalable, SSE-S3 encryption built-in
AI Provider	OpenAI GPT-4o (primary)	Strong instruction following for legal analysis
Auth	JWT (access + refresh tokens)	Stateless, scalable
PDF Parsing	pdf-parse / pdfjs-dist	Server-side text extraction
Deployment	Docker Compose → AWS ECS / Railway	Containerised, reproducible environments

B. Detailed Component Architecture

B.1 Component Responsibilities

Component	Responsibilities
AuthService	Login, logout, JWT issuance, refresh token rotation, witness token generation
UserService	CRUD users, assign roles, manage org membership
ContractService	Upload, parse, CRUD, status transitions, metadata management
FileService	S3 upload/download, signed URL generation, virus scan hook
AIService	Text chunking, prompt construction, OpenAI calls, response formatting
AuditService	Append-only event logging, retrieval for admin view
NotificationService	Expiry alerts, email triggers (future)
WitnessService	Secure link generation, single-contract scoped access

B.2 Communication Diagram

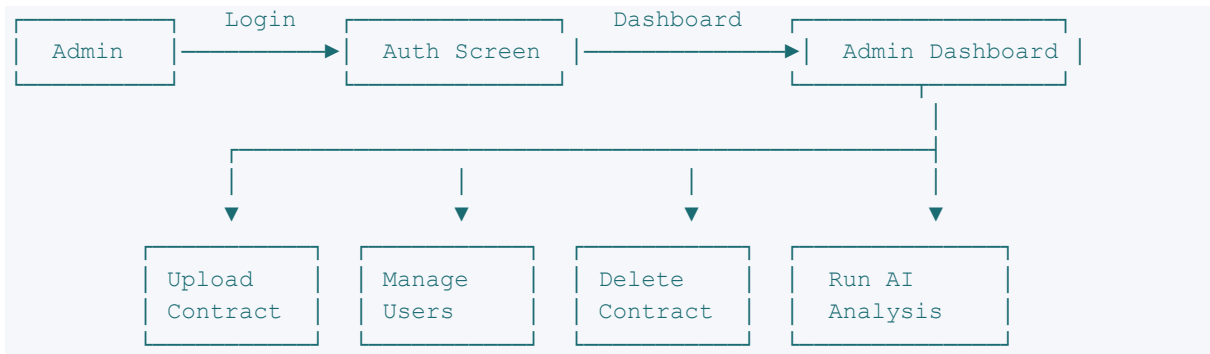


B.3 Data Flow: Contract Upload

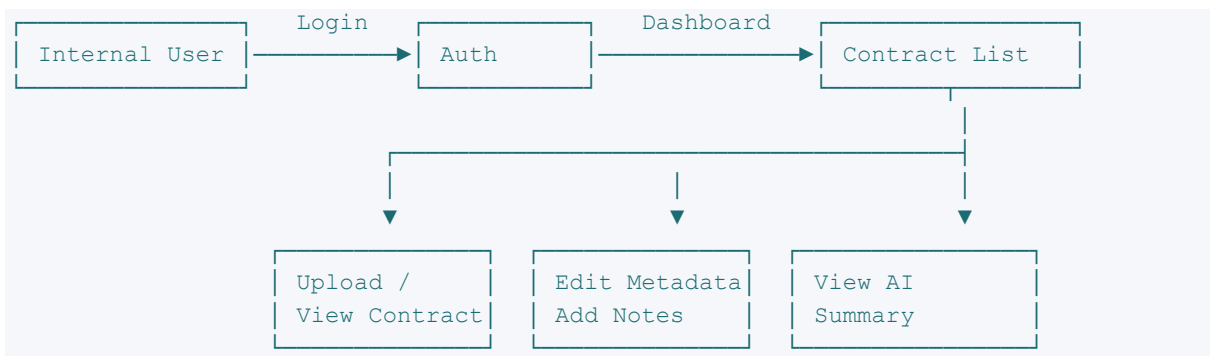
1. User uploads PDF via frontend
2. Frontend POSTs multipart/form-data to `/api/contracts`
3. `AuthMiddleware` validates JWT + checks role (Admin or Internal)
4. `ContractController` calls `FileService.upload()`
5. `FileService` stores file in S3, returns `fileKey` + `checksum`
6. `ContractService.parseText()` extracts text via `pdf-parse`
7. `ContractRepository.create()` persists contract + metadata
8. `AuditService.log()` records creation event
9. Response returns contract ID and parsed metadata

C. User Flow Diagrams

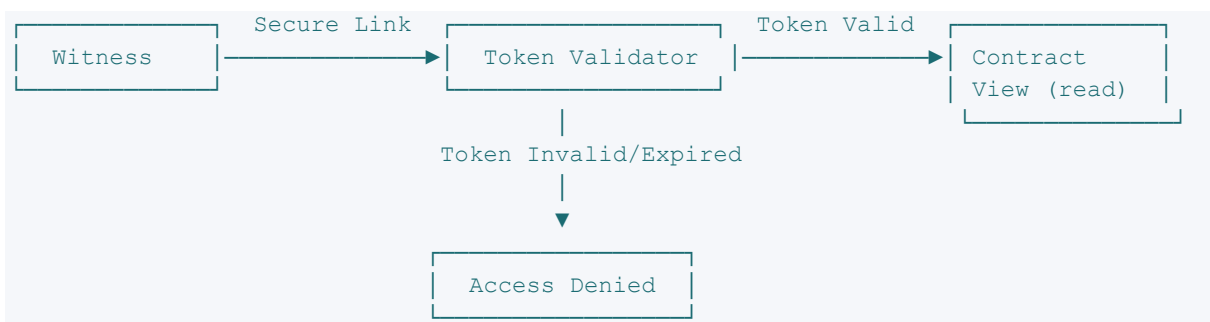
C.1 Administrator Flow



C.2 Internal User Flow



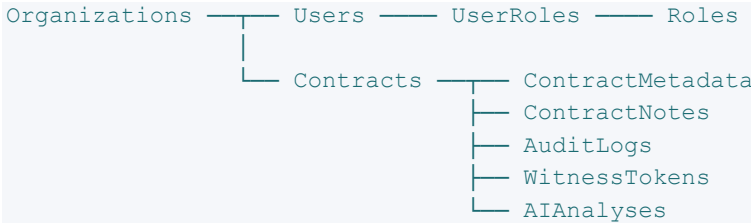
C.3 External Witness Flow



■ **Note:** Witness has no login; access is governed entirely by a one-time scoped JWT embedded in the link.

D. Database Design

D.1 Entity Relationship Overview



D.2 Table Definitions

organizations

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Tenant identifier
name	VARCHAR(255)	NOT NULL	Organization name
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	Creation timestamp
settings	JSONB	DEFAULT '{}'	Org-level config

users

Column	Type	Constraints	Description
id	UUID	PK	User identifier
org_id	UUID	FK → organizations.id, NOT NULL	Tenant reference
email	VARCHAR(320)	UNIQUE, NOT NULL	Login email
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash
full_name	VARCHAR(255)	NOT NULL	Display name
role	ENUM	NOT NULL (admin internal witness)	RBAC role
is_active	BOOLEAN	DEFAULT true	Soft disable
created_at	TIMESTAMPTZ	DEFAULT NOW()	Registration time
last_login	TIMESTAMPTZ		Last successful login

contracts

Column	Type	Constraints	Description
id	UUID	PK	Contract identifier
org_id	UUID	FK → organizations.id	Tenant scope
title	VARCHAR(500)	NOT NULL	Contract title
counterparty	VARCHAR(500)	NOT NULL	Legal counterparty name
status	ENUM	NOT NULL (Draft Active Expired Terminated)	Lifecycle status
tags	TEXT[]	DEFAULT '{}'	Searchable tags
expiration_date	DATE		Expiry date for alerts
file_key	VARCHAR(1000)	NOT NULL	S3 object key
file_checksum	VARCHAR(64)	NOT NULL	SHA-256 integrity check
extracted_text	TEXT		Parsed plain text
uploaded_by	UUID	FK → users.id	Uploader reference
created_at	TIMESTAMPTZ	DEFAULT NOW()	Upload timestamp
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last modification

contract_notes

Column	Type	Constraints	Description
id	UUID	PK	Note identifier
contract_id	UUID	FK → contracts.id	Parent contract
author_id	UUID	FK → users.id	Note author
content	TEXT	NOT NULL	Note body
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation timestamp

audit_logs

Column	Type	Constraints	Description
id	UUID	PK	Log entry ID
org_id	UUID	FK → organizations.id	Tenant scope

Column	Type	Constraints	Description
actor_id	UUID	FK → users.id, NULLABLE	User who acted
contract_id	UUID	FK → contracts.id, NULLABLE	Affected contract
action	VARCHAR(100)	NOT NULL	e.g. CONTRACT_UPLOADED
old_value	JSONB		Before-state snapshot
new_value	JSONB		After-state snapshot
ip_address	INET		Request IP
created_at	TIMESTAMPTZ	DEFAULT NOW()	Event timestamp

witness_tokens

Column	Type	Constraints	Description
id	UUID	PK	Token record ID
contract_id	UUID	FK → contracts.id	Scoped contract
token_hash	VARCHAR(255)	NOT NULL, UNIQUE	SHA-256 of raw token
issued_by	UUID	FK → users.id	Issuing admin
expires_at	TIMESTAMPTZ	NOT NULL	Token expiry
used_at	TIMESTAMPTZ		First access timestamp
is_revoked	BOOLEAN	DEFAULT false	Manual revocation

ai_analyses

Column	Type	Constraints	Description
id	UUID	PK	Analysis ID
contract_id	UUID	FK → contracts.id	Source contract
type	ENUM	NOT NULL (summary risk chat)	Analysis type
prompt_used	TEXT	NOT NULL	Full prompt sent to AI
response	TEXT	NOT NULL	AI response
model_version	VARCHAR(100)	NOT NULL	e.g. gpt-4o-2024-08
tokens_used	INTEGER		Billing tracking
created_at	TIMESTAMPTZ	DEFAULT NOW()	Analysis timestamp

Column	Type	Constraints	Description
created_by	UUID	FK → users.id	Requesting user

📌 **Note:** Row-Level Security (RLS) on PostgreSQL enforces org_id isolation. Every query automatically filters by the authenticated tenant's org_id via a SET LOCAL variable in the session.

E. Backend Design

E.1 Folder Structure

```
backend/
├── src/
│   ├── config/           # env, database, redis, s3 clients
│   ├── middleware/
│   │   ├── auth.ts       # JWT validation, role injection
│   │   ├── rbac.ts       # Role guard middleware factory
│   │   ├── audit.ts      # Automatic audit event capture
│   │   ├── errorHandler.ts
│   │   └── rateLimiter.ts
│   ├── modules/
│   │   ├── auth/
│   │   │   ├── auth.controller.ts
│   │   │   ├── auth.service.ts
│   │   │   └── auth.repository.ts
│   │   ├── contracts/
│   │   │   ├── contracts.controller.ts
│   │   │   ├── contracts.service.ts
│   │   │   └── contracts.repository.ts
│   │   ├── ai/
│   │   │   ├── ai.controller.ts
│   │   │   ├── ai.service.ts
│   │   │   └── prompts/
│   │   ├── users/
│   │   ├── files/
│   │   ├── audit/
│   │   └── witness/
│   ├── shared/
│   │   ├── types/
│   │   ├── utils/
│   │   └── constants/
│   └── app.ts             # Express app bootstrap
├── prisma/
│   └── schema.prisma
├── tests/
│   ├── unit/
│   └── integration/
└── Dockerfile
```

E.2 Service Layer Pattern

Each service follows the Single Responsibility Principle. Services never access the database directly — they delegate to a repository layer.

```
// contracts.service.ts (abbreviated)
class ContractService {
  constructor(
```

```
private repo: ContractRepository,
private fileService: FileService,
private auditService: AuditService,
) {}

async uploadContract(dto: UploadContractDto, actorId: string) {
  const fileKey = await this.fileService.upload(dto.file);
  const text = await this.fileService.extractText(fileKey);
  const contract = await this.repo.create({ ...dto, fileKey, text });
  await this.auditService.log('CONTRACT_UPLOADED', actorId, contract.id);
  return contract;
}
```

E.3 Repository Layer Pattern

Repositories wrap Prisma calls and enforce org-scoped queries. No raw SQL except for RLS-setting statements.

```
// contracts.repository.ts (abbreviated)
class ContractRepository {
  async findAll(orgId: string, filters: ContractFilters) {
    return prisma.contract.findMany({
      where: { orgId, ...filters },
      orderBy: { createdAt: 'desc' },
    });
  }
}
```

E.4 Middleware Stack

Middleware	Purpose	Applied To
helmet()	HTTP security headers	All routes
cors()	CORS with whitelist	All routes
rateLimiter	100 req/min per IP	All routes
authMiddleware	JWT validation, user injection	Protected routes
rbacGuard(role)	Role enforcement	Role-restricted routes
auditMiddleware	Log mutating requests	POST/PUT/DELETE routes
errorHandler	Centralized error formatting	App-level (last)

E.5 Authentication Flow

1. Client POSTs { email, password } to POST /api/auth/login
2. authService.login() verifies password against bcrypt hash
3. On success: issue short-lived access token (15m) + refresh token (7d)
4. Refresh token stored as HttpOnly cookie; access token returned in body
5. Client attaches access token as Authorization: Bearer <token>
6. authMiddleware validates JWT signature + expiry + checks blacklist in Redis
7. On expiry: POST /api/auth/refresh rotates the refresh token
8. Logout: refresh token added to Redis blacklist (TTL = remaining validity)

F. API Design

F.1 Auth Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/login	Public	Authenticate user, return JWT
POST	/api/auth/logout	Bearer	Invalidate refresh token
POST	/api/auth/refresh	Cookie	Rotate refresh token
POST	/api/auth/witness/:token	Public	Validate witness token

F.2 Contract Endpoints

Method	Endpoint	Auth	Description
GET	/api/contracts	Admin Internal	List all org contracts with filters
POST	/api/contracts	Admin Internal	Upload new contract (multipart or text)
GET	/api/contracts/:id	Admin Internal Witness*	Get contract details
PUT	/api/contracts/:id/metadata	Admin Internal	Update title, tags, status
DELETE	/api/contracts/:id	Admin only	Soft-delete contract
GET	/api/contracts/:id/file	Admin Internal	Get signed S3 download URL
GET	/api/contracts/:id/notes	Admin Internal	List notes for contract
POST	/api/contracts/:id/notes	Admin Internal	Add note to contract
GET	/api/contracts/:id/audit	Admin only	View audit trail for contract

🚩 **Note:** * Witness can access `GET /api/contracts/:id` only if their scoped token matches the contract ID.

F.3 AI Endpoints

Method	Endpoint	Auth	Description
POST	/api/ai/:contractId/summary	Admin Internal	Generate obligations/payment/termination summary
POST	/api/ai/:contractId/risk	Admin Internal	Run risk analysis on contract text
POST	/api/ai/:contractId/chat	Admin Internal	Clause investigator — single-turn Q&A

Method	Endpoint	Auth	Description
GET	/api/ai/:contractId/history	Admin Internal	Retrieve past AI analyses

F.4 User Management Endpoints

Method	Endpoint	Auth	Description
GET	/api/users	Admin only	List organization users
POST	/api/users	Admin only	Invite / create user
PUT	/api/users/:id/role	Admin only	Change user role
DELETE	/api/users/:id	Admin only	Deactivate user
POST	/api/users/witness-link	Admin only	Generate scoped witness link

F.5 Standard Request / Response Shapes

Successful Response

```
{
  "success": true,
  "data": { ... },
  "meta": { "timestamp": "2024-01-01T00:00:00Z" }
}
```

Error Response

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "You do not have permission to perform this action.",
    "statusCode": 403
  }
}
```

F.6 Standard Error Codes

HTTP Code	Error Code	Meaning
400	VALIDATION_ERROR	Request body failed schema validation
401	UNAUTHENTICATED	Missing or invalid JWT

HTTP Code	Error Code	Meaning
403	UNAUTHORIZED	Valid token but insufficient role
404	NOT_FOUND	Resource does not exist in this tenant
409	CONFLICT	Resource state conflict (e.g. duplicate email)
429	RATE_LIMITED	Too many requests
500	INTERNAL_ERROR	Server-side unexpected failure

G. Frontend Design

G.1 Page Hierarchy & Routing

/	→ Redirect to /dashboard
/login	→ LoginPage
/witness/:token	→ WitnessContractView (no auth required)
/dashboard	→ DashboardPage [Protected]
/contracts	→ ContractListPage [Protected]
/contracts/new	→ ContractUploadPage [Admin Internal]
/contracts/:id	→ ContractDetailPage [Protected]
/contracts/:id/ai	→ AIAalysisPage [Admin Internal]
/admin	→ AdminPage [Admin only]
/admin/users	→ UserManagementPage [Admin only]
*	→ NotFoundPage

G.2 Component Hierarchy

App
├─ AuthProvider (context)
├─ Router
│ ├─ ProtectedRoute (checks auth + role)
│ └─ WitnessRoute (checks witness token)
├─ Layout
│ ├─ Sidebar / Navbar
│ └─ <Outlet />
│ └─ DashboardPage
│ ├─ ExpiryAlertBanner
│ └─ ContractStatusSummary
│ └─ ContractListPage
│ ├─ FilterBar (status, tags, search)
│ └─ ContractTable
│ └─ ContractDetailPage
│ ├─ ContractMetadataPanel
│ ├─ ContractPDFViewer
│ ├─ NotesPanel (hidden from Witness)
│ └─ AIAalysisPanel (hidden from Witness)
│ └─ AIAalysisPage
│ ├─ AISummaryCard
│ ├─ RiskAnalysisCard
│ └─ ClauseInvestigatorChat

G.3 State Management Strategy

State Type	Tool	Rationale
Server State (API data)	React Query (TanStack Query)	Automatic caching, background refetch, loading states
Global UI State (auth, user)	Zustand store	Minimal boilerplate, no context hell
Form State	React Hook Form + Zod	Validation-first, type-safe schemas
Ephemeral UI (modals, tooltips)	Local useState	No need to globalise transient state

G.4 Role-Based UI Rendering

A `usePermission(role)` hook reads from the Zustand auth store and returns boolean permission flags. Components conditionally render controls based on these flags, ensuring UI and API authorization are consistent.

```
const { canDelete, canViewAI, canManageUsers } = usePermission();
{canDelete && <DeleteButton contractId={id} />}
{canViewAI && <AIAnalysisPanel contractId={id} />}
```

H. AI Module Design

H.1 PDF Text Extraction Workflow

1. Upload triggers `FileService.extractText(fileKey)`
2. File downloaded from S3 into memory buffer
3. `pdf-parse` extracts raw text; `pdfjs-dist` used as fallback for complex PDFs
4. Text cleaned: strip excessive whitespace, normalize line endings
5. Stored in `contracts.extracted_text` column
6. Character count checked; if $> 100,000$ chars, text is chunked for AI calls

H.2 Text Chunking Strategy

Large contracts are split into overlapping chunks of 6,000 tokens (50% overlap) to maintain clause context across chunk boundaries. For summary and risk analysis, chunks are processed sequentially and responses merged.

H.3 Prompt Engineering Strategy

All prompts follow a strict structure: System Role → Context Injection → Task Definition → Format Instructions → Safety Constraints.

H.3.1 Summary Prompt

```
SYSTEM:
You are a neutral contract analysis assistant. You analyze contract text
and summarize key provisions in plain English. You do NOT provide legal
advice. Always end your response with the disclaimer:
'DISCLAIMER: This is an AI-generated summary for informational purposes
only and does not constitute legal advice.'
```

```
USER:
Analyze the following contract and answer these three questions:
1. What are my obligations under this contract?
2. How and when do I get paid (or make payment)?
3. What are the termination conditions?

Base your answers ONLY on the contract text provided below.
If the contract does not address a question, state: 'Not specified.'
```

```
CONTRACT TEXT:
{extracted_text}
```

H.3.2 Risk Analysis Prompt

```
SYSTEM:
You are a neutral contract risk detector. Identify potentially
problematic clauses. Do NOT advise on whether to sign. Always
include the disclaimer about not constituting legal advice.

USER:
Scan this contract for the following risk categories:
- Auto-renewal clauses
- Unlimited or uncapped liability
- Aggressive or one-sided indemnification
- Unusual IP assignment clauses
- Non-compete or exclusivity restrictions
- Unusual payment clawback provisions

For each risk found: quote the relevant clause (max 2 sentences),
explain the risk in plain English, and rate severity: LOW / MEDIUM / HIGH.
If no risks found in a category, state 'None detected.'

CONTRACT TEXT:
{extracted_text}
```

H.3.3 Clause Investigator (Chat) Prompt

```
SYSTEM:
You are a contract clause investigator. Answer the user's question
using ONLY the contract text provided. If the answer is not in the
contract, respond: 'This contract does not address that topic.'
Do not speculate beyond the contract text. Include disclaimer.

USER:
CONTRACT TEXT:
{extracted_text}

QUESTION: {user_question}
```

H.4 AI Safety Constraints

Constraint	Implementation
No legal advice	Hard-coded system prompt instruction on every call
Legal disclaimer	Post-processing check: disclaimer appended if missing
Contract-only answers	System prompt instructs AI to cite only provided text
No PII leakage	Contract text never logged to application logs (only to ai_analyses table, encrypted)
Model pinning	Specific model version used (e.g. gpt-4o-2024-08-06) to prevent drift
Token budget	max_tokens capped at 2,000 per call to control cost and response scope

I. Security Design

I.1 RBAC Permission Matrix

Action	Admin	Internal	Witness
View contract list	✓	✓	X (single only)
Upload contract	✓	✓	X
Edit contract metadata	✓	✓	X
Delete contract	✓	X	X
View contract file	✓	✓	✓ (assigned only)
View contract notes	✓	✓	X
Add contract notes	✓	✓	X
Run AI analysis	✓	✓	X
View AI analysis	✓	✓	X
Manage users	✓	X	X
Generate witness link	✓	X	X
View audit log	✓	X	X
Change org settings	✓	X	X

I.2 JWT Strategy

Property	Access Token	Refresh Token
Expiry	15 minutes	7 days
Storage (Client)	Memory (JS variable)	HttpOnly cookie
Transport	Authorization header	Cookie header (SameSite=Strict)
Payload	userId, orgId, role, jti	userId, jti
Revocation	Redis blacklist (jti)	Redis blacklist (jti)
Rotation	—	On every refresh call

I.3 Password Storage Strategy

- Algorithm: bcrypt with cost factor 12
- Never store plaintext passwords
- Passwords never logged at any level
- Minimum password complexity enforced at API and frontend validation
- Rate limiting on /api/auth/login (5 attempts per 15 minutes per IP)

I.4 Witness Token Strategy

1. Admin calls POST /api/users/witness-link with { contractId, expiresIn }
2. Server generates cryptographically random 48-byte token (crypto.randomBytes)
3. SHA-256 hash of token stored in witness_tokens table
4. Raw token embedded in URL: /witness/{token}
5. On access: raw token hashed and compared against DB — no timing attack surface
6. Token can only access the one contract_id it was issued for
7. Token is single-use (used_at recorded on first access)
8. Token expiry enforced at DB query time

I.5 Encryption Strategy

Data	Method	Where
Files at rest	AWS S3 SSE-S3 (AES-256)	S3 buckets
Database at rest	PostgreSQL TDE or RDS encryption	DB volume
Transit	TLS 1.3 enforced	All endpoints
Sensitive env vars	AWS Secrets Manager / .env not committed	CI/CD
Password hashes	bcrypt (cost 12)	users table
Witness token	SHA-256 one-way hash	witness_tokens table

I.6 Audit Logging Strategy

- Audit log is append-only; no UPDATE or DELETE allowed on audit_logs
- Database trigger prevents modification of existing audit records
- Every data-mutating API action writes an audit event (actor, action, before/after)
- Witness access events are logged separately with token_id reference
- Logs are queryable by admin per contract or per user
- Logs are exportable as CSV (admin feature)

J. Development Roadmap

J.1 Milestone Overview

Milestone	Duration	Deliverable
M1: Foundation	Week 1–2	Project setup, CI/CD, DB schema, auth endpoints, role middleware
M2: Contract Core	Week 3–4	Upload, file storage, text extraction, metadata CRUD, audit log
M3: Frontend Shell	Week 3–4	React app scaffold, routing, auth flow, protected routes
M4: AI Module	Week 5–6	AI service integration, summary, risk, chat — with prompt safety
M5: Witness System	Week 5	Witness token generation, scoped access, witness view
M6: Dashboard & Polish	Week 6–7	Expiry alerts, filters, notes, UI refinement
M7: Security Hardening	Week 7	Penetration testing, rate limiting, RLS audit
M8: Testing & Demo	Week 8	Full test coverage, demo data, deployment, documentation

J.2 Sprint Plan (2-Week Sprints)

Sprint 1 (Weeks 1–2): Foundation

- Initialize monorepo, configure ESLint/Prettier/TypeScript
- Docker Compose: Postgres + Redis + App
- Prisma schema + initial migration
- Auth endpoints: register, login, logout, refresh
- JWT middleware + RBAC guard
- Frontend: Vite + React + Tailwind bootstrap, login page

Sprint 2 (Weeks 3–4): Contracts & Files

- S3 configuration + FileService
- PDF upload endpoint + text extraction
- Contract CRUD endpoints + metadata
- Audit logging service + DB trigger
- Frontend: Contract list, upload form, contract detail page
- React Query integration for data fetching

Sprint 3 (Weeks 5–6): AI + Witness

- OpenAI client integration + rate limiting
- Summary, risk analysis, and chat endpoints

- Frontend AI analysis page + chat interface
- Witness token generation endpoint
- Witness view (scoped, read-only)
- Notes system (backend + frontend)

Sprint 4 (Weeks 7–8): Polish + Security + Demo

- Expiry alert dashboard
- Admin user management UI
- Security audit: RBAC matrix test, token tests
- Integration tests for all major flows
- Deployment to staging environment
- Demo data seeding + documentation

K. Team Distribution

K.1 Developer Responsibilities

Developer	Primary Domain	Key Responsibilities
Developer 1	Backend & Security	Auth system, JWT/RBAC middleware, user management, audit log, witness token system, API design, security hardening, database schema, Prisma setup
Developer 2	Backend & AI	Contract service, file service, S3 integration, PDF parsing, AI service (prompt engineering, OpenAI integration), AI endpoints, text chunking
Developer 3	Frontend	React app architecture, all pages and components, React Query integration, Zustand auth store, role-based UI rendering, form validation, witness view

K.2 Shared Responsibilities

- Sprint planning and retrospectives (all three)
- API contract definition (Dev 1 + Dev 2 define, Dev 3 consumes)
- Code review (at minimum one reviewer per PR)
- Integration testing (Dev 1 + Dev 2 write API tests; Dev 3 writes E2E tests)
- Documentation: each developer documents their own modules

K.3 Dependency Management

To avoid blocking, use the following sequencing: Dev 1 delivers auth endpoints first (week 1), enabling Dev 3 to build the login flow. Dev 2 delivers the contract upload endpoint (end of week 3), enabling Dev 3 to build the upload UI. API contracts (TypeScript types) are shared via a `/shared/types` package agreed upon in Sprint 1.

L. GitHub Collaboration Strategy

L.1 Branching Model

Branch	Purpose	Protection Rules
main	Production-ready code only	Requires PR + 2 approvals + CI pass
develop	Integration branch; always deployable	Requires PR + 1 approval + CI pass
feature/DEV1-*	Individual feature work (Dev 1 prefix)	No direct push to main or develop
feature/DEV2-*	Individual feature work (Dev 2 prefix)	No direct push to main or develop
feature/DEV3-*	Individual feature work (Dev 3 prefix)	No direct push to main or develop
hotfix/*	Critical production fixes	Merges to main AND develop

L.2 Pull Request Workflow

- Developer creates feature branch from develop
- Implements feature with atomic commits (conventional commits format)
- Pushes branch and opens PR against develop
- PR template requires: description, test coverage confirmation, screenshot (frontend), security checklist
- At least one other developer reviews (two for security-sensitive code)
- CI pipeline must pass (lint, type-check, unit tests)
- Squash-merge into develop with descriptive commit message

L.3 Commit Convention

```
feat(auth): add refresh token rotation
fix(ai): handle empty extracted text gracefully
chore(db): add index on contracts.org_id
test(witness): add token expiry integration test
docs(api): document error response format
```

L.4 CI/CD Pipeline

- On PR open: lint + typecheck + unit tests
- On merge to develop: integration tests + Docker build
- On merge to main: full test suite + build + deploy to staging

4. Environment secrets managed via GitHub Secrets — never in code

M. Risks and Challenges

Risk	Category	Severity	Mitigation
AI hallucinations producing inaccurate contract summaries	AI	HIGH	Mandatory disclaimer on all AI output; instruct AI to cite only provided text; do not use AI output for binding decisions
Witness link leaked or forwarded	Security	HIGH	Single-use tokens; short expiry (48h max); revocation endpoint; access logged
Tenant data isolation failure (cross-org data access)	Security	CRITICAL	PostgreSQL RLS; orgId enforced at repository layer; automated tests asserting cross-tenant isolation
PDF parsing failure or malformed input	Technical	MEDIUM	Fallback parser; graceful error with manual text-paste option; contract still accessible even if text extraction fails
OpenAI API latency or outage	Integration	MEDIUM	Async AI job queue (future); user-facing loading states; retry logic with exponential backoff
Large contract files (>50MB) causing timeouts	Technical	MEDIUM	File size limit enforced (25MB); streaming upload to S3; async text extraction after upload
JWT token theft via XSS	Security	HIGH	Access token in memory only (not localStorage); strict CSP headers; HttpOnly refresh cookie
Developer merge conflicts on shared schema	Technical	LOW	Prisma migration files reviewed in PRs; one developer owns schema merges

N. MVP vs Advanced Features

N.1 Minimum Viable Product (MVP)

These features are required for a functional, demonstrable system and must be completed within the 8-week timeline.

Feature	Module
Email + password authentication with JWT	Auth
Three roles: Admin, Internal, Witness	RBAC
PDF and text-paste contract upload	Contracts
S3 file storage with signed download URLs	Files
PDF text extraction	AI Prerequisite
Contract metadata (title, status, tags, expiry, counterparty)	Contracts
Audit log (append-only, per contract)	Audit
Contract list with status + tag filters	Dashboard
AI summary (obligations, payment, termination)	AI
AI risk analysis	AI
Clause investigator chat (single-turn)	AI
Internal notes per contract	Workspace
Scoped witness access via secure link	Witness
Expiry alert banner on dashboard	Dashboard
Admin: user management (invite, role change)	Admin

N.2 Advanced / Nice-to-Have Features

These features enhance the product but are out of scope for the MVP sprint.

Feature	Value
E-signature integration (DocuSign / HelloSign API)	Allows binding contract execution within the platform
Email notifications for expiry alerts	Proactive lifecycle management without logging in
Multi-turn AI conversation with memory	Deeper clause investigation across complex contracts
Contract templates library	Accelerate new contract drafting
Version history (contract redlines)	Track changes between contract versions

Feature	Value
Bulk export (all contracts as ZIP)	Data portability for compliance
SSO / SAML / OAuth (Google, Microsoft)	Enterprise authentication convenience
Webhook notifications to external systems	Integration with Slack, Zapier, etc.
Mobile-responsive PWA	Access on mobile devices
AI-powered clause comparison (two contracts)	Side-by-side risk delta